



INSTITUTO SUPERIOR POLITÉCNICO DA CATEPA

ALGORITMO E ESTRUTURA DE DADOS II

- PROJECTO: Estoque de vendas

CURSO: Engenharia Informática

GRUPO N.º 2

TURMA: Única

PERÍODO: Manhã

DOCENTE

Osvaldo Queta, Lic.

Ano lectivo 2025/26

INSTITUTO SUPERIOR POLITÉCNICO DA CATEPA

ALGORITMO E ESTRUTURA DE DADOS II

PROJECTO: Estoque de vendas

GRUPO N.º 2

AUTORES

Nº	NOME	CLASSIF.
01	António Bravo	
02	Felícia Campos	
03	Jorge Cabaça	
01	Santo Moreira	

ÍNDICE

1. DESCRIÇÃO GERAL DO PROJECTO.....	4
2. DOCUMENTAÇÃO E JUSTIFICAÇÃO DAS DECISÕES DO CÓDIGO	5
3. EXPLICAÇÃO DOS PASSOS MENOS ÓBVIOS, IMPORTANTES OU ORIGINAIS DO CÓDIGO.....	11
4. TESTES E RESULTADOS RELEVANTES.....	13
5. IMPLEMENTAÇÃO DO PROGRAMA	14

1. DESCRIÇÃO GERAL DO PROJECTO

O presente projecto consiste no desenvolvimento de um sistema de gestão de produtos com interface gráfica, criado em Python utilizando a biblioteca CustomTkinter. O objectivo principal é facilitar o controlo de stock de uma loja ou armazém, permitindo o registo, actualização, remoção e consulta de produtos de forma simples, organizada e eficiente.

O sistema permite cadastrar produtos por categorias pré-definidas, atribuindo automaticamente um identificador único para cada item. Para cada produto são registadas informações como nome, categoria, preço, quantidade em stock e limite mínimo. A aplicação apresenta uma listagem completa dos produtos, bem como filtros por nome e ID para facilitar a pesquisa e a gestão.

Além disso, o projecto inclui funcionalidades de controlo automático de stock, emitindo alertas sempre que a quantidade de um produto atinge ou fica abaixo do limite mínimo definido. O sistema também calcula e apresenta a quantidade total de produtos e o valor total em stock, permitindo uma visão geral do inventário.

A interface gráfica foi desenvolvida para ser intuitiva e de fácil utilização, proporcionando uma experiência prática ao utilizador. Este projecto tem como finalidade apoiar a organização e o controlo de produtos em pequenos negócios ou ambientes académicos, demonstrando a aplicação de conceitos de programação, interface gráfica e gestão de dados em memória.

2. DOCUMENTAÇÃO E JUSTIFICAÇÃO DAS DECISÕES DO CÓDIGO

2.1. Importação das bibliotecas

```
import customtkinter as ctk  
  
from tkinter import messagebox
```

Justificação:

Foi utilizada a biblioteca **CustomTkinter** para criar uma interface gráfica moderna e mais atractiva do que o Tkinter tradicional. O messagebox do Tkinter foi importado para apresentar mensagens ao utilizador (avisos, erros e confirmações), melhorando a interacção e a experiência de uso.

2.2. Configuração da aparência

```
ctk.set_appearance_mode("Dark")  
  
ctk.set_default_color_theme("green")
```

Justificação:

Definiu-se o modo escuro (Dark) para tornar a interface visualmente confortável e moderna. O tema verde foi escolhido por ser associado a sistemas de gestão e controlo, transmitindo organização e clareza.

2.3. Estrutura de dados para produtos

```
produtos = []
```

Justificação:

Foi utilizada uma lista em memória para armazenar os produtos. Cada produto é representado por um dicionário, permitindo guardar várias propriedades (nome, preço, quantidade, etc.). Esta decisão simplifica o projecto e evita a necessidade de base de dados externa.

2.4. Categorias pré-definidas

```
categorias = ["Alimentos", "Bebidas", "Higiene", "Eletrónica",  
"Roupas"]
```

Justificação:

Foram criadas categorias fixas para padronizar os registos e evitar erros de digitação. Facilita a organização e permite gerar IDs automáticos por categoria.

2.5. Função gerar ID automático

```
def gerar_id(categoria):  
    total = sum(1 for p in produtos if p["categoria"] ==  
categoria) + 1  
  
    return f"{categoria[:2].upper()}-{total:03d}"
```

Justificação:

Cria um identificador único por categoria. Exemplo: AL-001 (Alimentos). Isso facilita a identificação e pesquisa dos produtos. O uso de contagem automática evita repetição de IDs.

2.6. Função carregar (actualizar ecrã)

```
def carregar():
```

Responsável por:

- 🗑 Limpar a lista.
- 🔍 Filtrar produtos.
- 📄 Mostrar dados.
- 🧮 Calcular totais.

Justificação:

Centraliza a actualização da interface. Sempre que um produto é adicionado, removido ou actualizado, a função recarrega a lista para manter a informação actual.

Filtros

```
if filtro_nome.get().lower() not in p["nome"].lower():  
  
Permite pesquisar por nome.  
  
if filtro_id.get() and filtro_id.get() != p["id"]:
```

Permite filtrar por ID exacto.

Justificação:

Facilita a localização rápida de produtos num stock grande.

Alerta de stock baixo

```
aviso = "STOCK BAIXO" if p["quantidade"] <= p["limite"] else ""
```

Mostra aviso automático.

Justificação:

Ajuda no controlo de reposição de produtos.

Cálculo de totais

```
total_qtde += p["quantidade"]  
total_valor += p["quantidade"] * p["preco"]
```

Justificação:

Permite visão geral do stock e valor financeiro.

2.7. Função adicionar produto

```
def adicionar():
```

Responsável por inserir novos produtos.

Validação da categoria

```
if not categoria or categoria == "Selecione Categoria":
```

Justificação:

Evita registos incompletos.

Criação do dicionário produto

```
produto = {  
    "id": gerar_id(categoria),  
    "nome": nome_entry.get(),  
    ...  
}
```

Justificação:

Agrupa todos os dados num único objecto estruturado.

Mensagens ao utilizador

```
messagebox.showinfo("Sucesso", ...)
```

Justificação:

Confirma operações e melhora a experiência.

Alerta automático

Verifica stock mínimo após adicionar.

Justificação:

Evita ruptura de stock.

2.8. Função actualizar produto

```
def actualizar():
```

Actualiza dados com base no ID.

Justificação:

Permite corrigir preços, quantidades ou nomes sem apagar o produto.

Pesquisa por ID

```
if p["id"] == pid:
```

Justificação:

ID é único, evitando erros.

2.9. Função remover produto

```
def remover():
```

Remove produto pelo ID.

Confirmação

```
confirm = messagebox.askyesno(...)
```

Justificação:

Evita eliminação acidental.

2.10. Função seleccionar produto

```
def seleccionar() :
```

Permite clicar num produto da lista e carregar nos campos.

Justificação:

Facilita edição rápida sem digitar o ID manualmente.

2.11. Função limpar campos

```
def limpar_campos() :
```

Limpa entradas após operações.

Justificação:

Evita dados antigos permanecerem no formulário.

2.12. Criação da janela principal

```
app = ctk.CTk()
```

Define janela principal.

```
app.geometry("940x650")
```

Tamanho fixo para manter layout organizado.

2.13. Interface gráfica

Foram adicionados:

-  Título
-  Campos de filtro
-  Lista de produtos
-  Campos de entrada
-  Botões de acção
-  Totais
-  Rodapé

Justificação:

Organização visual clara e intuitiva. Cada botão executa uma função específica do sistema.

2.14. Rodapé com desenvolvedores

```
rodape = ctk.CTkLabel(...)
```

Justificação:

Identifica os autores do projecto para fins académicos.

2.15. Execução do sistema

```
carregar()  
app.mainloop()
```

Justificação:

carregar() mostra dados iniciais.mainloop() mantém a aplicação em execução.

Conclusão da justificação

As decisões tomadas visaram criar um sistema:

- 🚦 Simples de usar
- 🚦 Visualmente organizado
- 🚦 Funcional para gestão de stock
- 🚦 Sem necessidade de base de dados externa
- 🚦 Adequado para aprendizagem académica e uso em pequenos negócios

O projecto demonstra aplicação prática de Python com interface gráfica, estrutura de dados e lógica de controlo de stock.

3. EXPLICAÇÃO DOS PASSOS MENOS ÓBVIOS, IMPORTANTES OU ORIGINAIS DO CÓDIGO

3.1. Geração automática de ID por categoria: A função `gerar_id(categoria)` é uma das partes menos óbvias do código. Ela conta quantos produtos já existem dentro da mesma categoria e gera um identificador sequencial com base nisso. Por exemplo, se for o terceiro produto da categoria Alimentos, o ID será **AL-003**.

Esta decisão é importante porque:

- 🚦 Evita repetição de IDs;
- 🚦 Organiza os produtos por categoria;
- 🚦 Facilita a identificação e pesquisa.

É uma solução original por funcionar sem base de dados.

3.2. Sistema de filtragem dentro da função `carregar()`: Antes de mostrar os produtos, o sistema verifica se há filtros activos por nome ou por ID. A linha:

```
if filtro_nome.get().lower() not in p["nome"].lower():
```

permite pesquisar apenas uma parte do nome, ignorando maiúsculas e minúsculas.

Isto não é óbvio para iniciantes, mas é importante porque torna a pesquisa mais flexível e funcional.

3.3. Alerta automático de stock baixo: O código verifica automaticamente se a quantidade do produto é menor ou igual ao limite mínimo definido:

```
p["quantidade"] <= p["limite"]
```

Quando isso acontece, surge aviso visual e mensagem.

Esta funcionalidade destaca-se porque simula um sistema real de gestão de stock e funciona de forma automática após adicionar ou actualizar um produto.

3.4. Cálculo automático do valor total do stock: Durante a actualização da lista, o sistema soma:

- 🚦 Quantidade total;
- 🚦 Valor total em dinheiro.

O valor total é calculado multiplicando quantidade pelo preço de cada produto. Este cálculo é feito em tempo real sempre que a lista é actualizada.

Esta decisão é importante porque permite ao utilizador ter uma visão financeira imediata do stock.

3.5. Função seleccionar() para edição automática A função seleccionar() permite clicar numa linha da lista e carregar os dados do produto nos campos de edição. O código extrai o ID do texto da linha seleccionada e procura o produto correspondente.

Esta parte não é óbvia porque envolve:

- ✚ Leitura de texto da interface;
- ✚ Separação da string pelo símbolo " | ";
- ✚ Preenchimento automático dos campos.

É uma funcionalidade típica de sistemas profissionais e melhora muito a usabilidade.

6. Uso de try/except para evitar falhas: Algumas funções usam try/except para impedir que o programa encerre caso o utilizador introduza dados inválidos (por exemplo letras no preço).

Isto é importante porque:

- ✚ Torna o sistema mais estável;
- ✚ Evita erros fatais;
- ✚ Permite mostrar mensagens amigáveis ao utilizador.

4. TESTES E RESULTADOS RELEVANTES

Durante o desenvolvimento do sistema de gestão de produtos, foram realizados diversos testes com o objectivo de verificar o correcto funcionamento das funcionalidades implementadas e garantir a estabilidade da aplicação.

4.1. Teste de inserção de produtos: Foram adicionados vários produtos com diferentes categorias, preços e quantidades. Verificou-se que o sistema gera automaticamente o ID de forma correcta e sequencial por categoria. Os produtos inseridos passaram a ser exibidos imediatamente na lista, confirmando o funcionamento da actualização automática do ecrã.

Resultado: Inserção de dados funcional e sem conflitos de identificação.

4.2. Teste de actualização de produtos: Foram seleccionados produtos já existentes e alterados os seus dados (nome, preço e quantidade). Após a actualização, os dados foram reflectidos correctamente na listagem.

Resultado: A função de actualização demonstrou precisão e resposta imediata.

4.3. Teste de remoção de produtos: Foram removidos produtos utilizando o ID. O sistema solicitou confirmação antes da eliminação e, após confirmação, removeu o produto da lista.

Resultado: Remoção segura e eficaz, evitando eliminações acidentais.

4.4. Teste de filtros por nome e ID Foram realizados testes utilizando diferentes palavras-chave e identificadores. O sistema filtrou correctamente os produtos, mostrando apenas os que correspondem aos critérios definidos.

Resultado: Sistema de pesquisa funcional, rápido e preciso.

4.5. Teste de alerta de stock mínimo: Foram inseridos produtos com quantidade igual ou inferior ao limite mínimo. O sistema apresentou aviso visual de stock baixo e mensagem de alerta.

Resultado: Funcionalidade de controlo de stock validada com sucesso.

4.6. Teste de cálculo de totais: Foram adicionados vários produtos com diferentes preços e quantidades. O sistema calculou automaticamente a quantidade total e o valor total em stock.

Resultado: Cálculos correctos e actualizados em tempo real.

5. IMPLEMENTAÇÃO DO PROGRAMA

```
import customtkinter as ctk
from tkinter import messagebox

ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("green")

# Lista de produtos em memória
produtos = []

# Categorias pré-definidas
categorias = ["Alimentos", "Bebidas", "Higiene", "Eletrónica",
"Roupas"]

# Gerar ID automático por categoria
def gerar_id(categoria):
    total = sum(1 for p in produtos if p["categoria"] ==
categoria) + 1
    return f"{categoria[:2].upper()}-{total:03d}"

# Atualizar ecrã
def carregar():
    lista.delete("1.0", "end")
    total_qtde = 0
    total_valor = 0.0

    for p in produtos:
        if filtro_nome.get().lower() not in p["nome"].lower():
            continue
        if filtro_id.get() and filtro_id.get() != p["id"]:
            continue

        aviso = " STOCK BAIXO" if p["quantidade"] <= p["limite"]
else ""

        lista.insert(
            "end",
            f'{p["id"]} | {p["nome"]} | {p["categoria"]} | '
            f'Preço: {p["preco"]:.2f} Kz | Qtde:
{p["quantidade"]}{aviso}\n'
        )

        total_qtde += p["quantidade"]
        total_valor += p["quantidade"] * p["preco"]

    totais.configure(
        text=f"Quantidade Total: {total_qtde} | Valor Total:
{total_valor:.2f} Kz"
    )
```

```

# Adicionar produto
def adicionar():
    try:
        categoria = categoria_combo.get()
        if not categoria or categoria == "Selecione Categoria":
            messagebox.showwarning("Aviso", "Selecione uma categoria!")
            return

        produto = {
            "id": gerar_id(categoria),
            "nome": nome_entry.get(),
            "categoria": categoria,
            "preco": float(preco_entry.get()),
            "quantidade": int(qtde_entry.get()),
            "limite": int(limite_entry.get())
        }

        produtos.append(produto)
        carregar()
        limpar_campos()
        messagebox.showinfo("Sucesso", f'Produto {produto["id"]} adicionado!')

        # Alerta automático se stock baixo
        if produto["quantidade"] <= produto["limite"]:
            messagebox.showwarning(
                "Atenção",
                f'O produto {produto["nome"]} atingiu o limite mínimo de stock!'
            )

    except:
        messagebox.showerror("Erro", "Dados inválidos!")

# Atualizar produto
def atualizar():
    pid = filtro_id.get().strip()
    if not pid:
        messagebox.showwarning("Aviso", "Informe o ID para atualizar!")
        return

    for p in produtos:
        if p["id"] == pid:
            p["nome"] = nome_entry.get()
            p["categoria"] = categoria_combo.get()
            p["preco"] = float(preco_entry.get())
            p["quantidade"] = int(qtde_entry.get())
            p["limite"] = int(limite_entry.get())

```

```

        carregar()
        limpar_campos()
        messagebox.showinfo("Sucesso", "Produto
atualizado!")

        # Alerta automático se stock baixo
        if p["quantidade"] <= p["limite"]:
            messagebox.showwarning(
                "Atenção",
                f'O produto {p["nome"]} atingiu o limite
mínimo de stock!'
            )
        return

        messagebox.showerror("Erro", "Produto não encontrado!")

# Remover produto
def remover():
    pid = filtro_id.get().strip()
    if not pid:
        messagebox.showwarning("Aviso", "Informe o ID para
remover!")
        return

    for i, p in enumerate(produtos):
        if p["id"] == pid:
            confirm = messagebox.askyesno("Confirmar",
f"Remover produto {pid}?")
            if confirm:
                produtos.pop(i)
                carregar()
                limpar_campos()
            return

        messagebox.showerror("Erro", "Produto não encontrado!")

# Selecionar produto da lista
def selecionar():
    try:
        linha = lista.get("sel.first", "sel.last").strip()
        if not linha:
            messagebox.showwarning("Aviso", "Selecione uma
linha da lista!")
            return
        pid = linha.split(" | ")[0]
        filtro_id.set(pid)

        for p in produtos:
            if p["id"] == pid:
                nome_entry.set(p["nome"])
                categoria_combo.set(p["categoria"])

```

```

        preco_entry.set(str(p["preco"]))
        qtde_entry.set(str(p["quantidade"]))
        limite_entry.set(str(p["limite"]))
        return

    except:
        messagebox.showwarning("Aviso", "Selecione corretamente
um produto!")

# Limpar campos
def limpar_campos():
    nome_entry.set("")
    categoria_combo.set("Selecione Categoria")
    preco_entry.set("")
    qtde_entry.set("")
    limite_entry.set("")
    filtro_id.set("")

# =====
# JANELA PRINCIPAL
# =====

app = ctk.CTk()
app.title("Gestão de Produtos")
app.geometry("940x650")
app.resizable(False, False)

titulo = ctk.CTkLabel(app, text="Gestão de Produtos",
font=("Arial", 26, "bold"))
titulo.place(x=320, y=10)

# Filtros
filtro_nome = ctk.CTkEntry(app, width=200,
placeholder_text="Filtrar por nome")
filtro_nome.place(x=20, y=70)

filtro_id = ctk.CTkEntry(app, width=150,
placeholder_text="Filtrar por ID")
filtro_id.place(x=240, y=70)

ctk.CTkButton(app, text="Filtrar", width=120,
command=carregar).place(x=410, y=70)

# Listagem
lista = ctk.CTkTextbox(app, width=900, height=250)
lista.place(x=20, y=110)

# Campos de entrada
nome_entry = ctk.CTkEntry(app, width=200,
placeholder_text="Nome do produto")
nome_entry.place(x=20, y=380)

```

```

categoria_combo = ctk.CTkComboBox(app, values=categorias,
width=150)
categoria_combo.set("Selecione Categoria")
categoria_combo.place(x=240, y=380)

preco_entry = ctk.CTkEntry(app, width=100,
placeholder_text="Preço")
preco_entry.place(x=410, y=380)

qtde_entry = ctk.CTkEntry(app, width=100,
placeholder_text="Quantidade")
qtde_entry.place(x=530, y=380)

limite_entry = ctk.CTkEntry(app, width=120,
placeholder_text="Limite mínimo")
limite_entry.place(x=650, y=380)

# Botões
ctk.CTkButton(app, text="Adicionar Produto", width=160,
command=adicionar).place(x=20, y=440)
ctk.CTkButton(app, text="Atualizar Produto", width=160,
command=atualizar).place(x=200, y=440)
ctk.CTkButton(app, text="Remover Produto", width=160,
command=remover).place(x=380, y=440)
ctk.CTkButton(app, text="Editar Selecionado", width=160,
command=selecionar).place(x=560, y=440)
ctk.CTkButton(app, text="Limpar Campos", width=140,
command=limpar_campos).place(x=740, y=440)

# Totais
totais = ctk.CTkLabel(app, text="", font=("Arial", 14, "bold"))
totais.place(x=20, y=520)

# Rodapé com os nomes dos desenvolvedores
rodape = ctk.CTkLabel(
    app,
    text="Desenvolvedores: António Bravo, Felícia Campos, Jorge
Cabaça, Santo Moreira",
    font=("Arial", 12),
    text_color="gray"
)
rodape.place(x=20, y=600)

carregar()
app.mainloop()

```